Meshtastic Node Monitor

Project Brief • W4DBR • March 2026

Project Overview

A locally hosted application that interfaces with a Meshtastic Node over a TCP connection where it continually gathers and stores that data, while another process serves up live monitor pages in a browser window, with a settings page for management.

Conceptual Overview

We'll start with building and hosting the app locally on a MacBook Pro M4+, and using a Chrome browser to display the Node Monitor interface. Ultimately, the data capture and display panel management will be ported to a scaled down e-ink display and housed in a desktop or wall mounted enclosure. The E-Ink display may or may not be color, so a switchable CSS style sheet should be incorporated and tested. The visual appearance should be that of a monitor panel or a system gauge, able to quickly convey the state of the local mesh; how much traffic there is, number of nodes id'ed, loudest nodes, closest nodes, busiest nodes, etc. I could see this being used mag-mounted to the side of my truck at a base camp where we might be tracking working crews. Or just monitoring a 300 node mesh to see who and where the big users are. Eventually would add capability to analyze mesh data captured while moving, so this collection of data is not centralized but full of perspective. This would imply that a gps coordinate of where the packet is received be part of the data capture.

Analyzing the packet paths, hops, and dead ends would be valuable to see how well a mesh is working. Trace routes on every packet could provide the links to build a visual network of interconnected nodes. Suggest a path for data collection, analysis, and presentation for this type of analysis.

Mesh Monitor Display Screen

- Project folder:
    - NodeMonitor/NMDisplay

The display is divided into three zones with movable borders

Left Panel (~60% width) — Map

- OSM map tiles rendered from local storage (fetched over HTTP if not present)
- Node dots plotted by GPS coordinates — red dots for active nodes
- Signal strength indicator next to each node dot
- Road and geography lines from map tile data
- Configurable zoom level, default to keeping all nodes in the list visible.

Right Panel (~40% width) — Telemetry Lists

A configurable set of telemetry lists, stacked vertically. Which lists appear, their order, and how many nodes each shows are all user-configurable via the web settings UI. Default selection is any four of the six.

Available Telemetry Lists

- LAST HEARD — nodes sorted by most recent packet received
- NEARBY — nodes sorted by GPS distance from your position
- BIG SIG — nodes sorted by strongest RSSI / SNR
- MOST PKTS — nodes sorted by total packets received from them
- MSGS SENT — nodes sorted by outbound message count
- MSGS RCVD — nodes sorted by inbound message count

Bottom Strip — Packet Activity Timeline

- Rolling bar chart histogram of packets per minute
- Time axis scrolls left — oldest data exits the left edge
- Clock Time markers along bottom axis (e.g. 11am, 12pm, 1pm, 2pm, 3pm, …)
- Regular packet bars in blue; message traffic accented in red
- Configurable time scale: 1 hour, 6 hours, or 24 hours of history
- Configurable Y-axis: auto-scale or fixed maximum pkt/min
- Data stored in rolling in-memory buffer; persisted to disk for reboot continuity
-

MeshMonitorDataCollector

- Subscribes to, and listens on the TCP network for Meshtastic packet broadcasts.
- Categorizes and saves them in a database
- Project folder:
    - NodeMonitor/NMListener
- Database Folder
    - NodeMonitor/NMData

Source of Meshtastic packet stream

- Heltec ESP32-S3 V4 running Meshtastic firmware
- Connected to home WiFi; TCP interface on 192.168.1.25 port 4403
- Subscribe to all Meshtastic packet callbacks
- Maintains locally hosted database tables as data store: node list, packet counts, message counts, timeline buffer

TCPInterface connects directly to port 4403. Event-driven callback model — subscribe once, receive all packet types. Packets differentiated by portnum field:

- TEXT_MESSAGE_APP — incoming/outgoing messages

- TELEMETRY_APP — battery, environment sensor data
- NODEINFO_APP — node identity, hardware info
- POSITION_APP — GPS coordinates for map plotting
- Unrecognized portnums counted toward total packet statistics

Settings System

All settings stored locally in config.json. Settings persist across reboots. A

Telemetry List Settings

- Which lists to display (choose any subset of the 6 available)
- Display order of selected lists
- Number of nodes to show per list (1–5 per list, independent)

Timeline Settings

- Time scale: 1 hour / 6 hours / 24 hours
- Y-axis mode: auto-scale or fixed maximum packets/minute
- Message traffic accent: on / off

Map Settings

- Zoom level
- Center mode: my node / recent activity centroid
- Node labels: show short name / dots only

Build Phases

Interface with the Meshtastic device and log the node data

- Connect TCPInterface to static IP
- Print raw packets to console — verify all portnum types flowing
- Build data store classes: NodeStore, PacketTimeline, MessageCounter
- Confirm GPS position data available for map plotting
- Commit data to database

Display the rendered live data in a browser window

- Build map tile loader and renderer (OSM tiles from local storage)
- Plot node dots at GPS-derived pixel coordinates

- Render telemetry list panels with live data
- Render packet timeline bar chart
- Full layout integration test

Settings & Web UI

- Build config.json schema and defaults
- Test all configurable parameters end-to-end